

Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе №4

Вычислительная математика

Вариант 8

Выполнил:
Караганов Павел

Группа: Р3210

Проверил:
Рыбаков Степан Дмитриевич

г. Санкт-Петербург,

2026

1 Цель лабораторной работы

Построить линейную и квадратичную аппроксимации заданной табличной функции методом наименьших квадратов, оценить качество приближения по среднеквадратичному отклонению, коэффициенту корреляции Пирсона и коэффициенту детерминации, а также выбрать наилучшую аппроксимацию.

2 Рабочие формулы метода

Для точек $(x_i, y_i), i = 1, 2, \dots, n$, линейная аппроксимация имеет вид

$$\varphi_1(x) = a_0 + a_1x.$$

Коэффициенты линейной модели находятся из нормальных уравнений:

$$\begin{cases} na_0 + (\sum x_i)a_1 = \sum y_i \\ (\sum x_i)a_0 + (\sum x_i^2)a_1 = \sum x_i y_i \end{cases}.$$

Квадратичная аппроксимация задаётся формулой

$$\varphi_2(x) = b_0 + b_1x + b_2x^2,$$

а её коэффициенты определяются системой

$$\begin{cases} nb_0 + (\sum x_i)b_1 + (\sum x_i^2)b_2 = \sum y_i \\ (\sum x_i)b_0 + (\sum x_i^2)b_1 + (\sum x_i^3)b_2 = \sum x_i y_i \\ (\sum x_i^2)b_0 + (\sum x_i^3)b_1 + (\sum x_i^4)b_2 = \sum x_i^2 y_i \end{cases}.$$

Среднеквадратичное отклонение вычисляется по формуле

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (\varphi(x_i) - y_i)^2}.$$

Коэффициент корреляции Пирсона для линейной зависимости:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}.$$

Коэффициент детерминации:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \varphi(x_i))^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

3 Вычислительная часть

$$f(x) = \frac{3x}{x^4 + 8}, \quad x \in [-2, 0], \quad h = 0.2.$$

Получаем 11 узлов табулирования.

3.1 Таблица значений функции

x_i	y_i
-2.0	-0.250000
-1.8	-0.291930
-1.6	-0.329815
-1.4	-0.354682
-1.2	-0.357370
-1.0	-0.333333
-0.8	-0.285388
-0.6	-0.221413
-0.4	-0.149522
-0.2	-0.074985
0.0	0.000000

3.2 Линейная аппроксимация

Ищем приближение вида $\varphi_1(x) = a_0 + a_1x$.

Для исходных данных получены суммы:

$$n = 11, \quad \sum x_i = -11.0, \quad \sum x_i^2 = 15.4, \quad \sum x_i y_i = 3.247874, \quad \sum y_i = -2.648438.$$

После решения уравнений:

$$a_0 = -0.104532, \quad a_1 = 0.136235.$$

Следовательно,

$$\varphi_1(x) = -0.104532 + 0.136235x.$$

Коэффициент корреляции Пирсона:

$$r = 0.758575, \quad R_1^2 = 0.575435.$$

По значению R_1^2 линейная аппроксимация является удовлетворительной.

Среднеквадратичное отклонение:

$$\sigma_1 = 0.074011.$$

3.3 Квадратичная аппроксимация

Ищем приближение вида $\varphi_2(x) = b_0 + b_1x + b_2x^2$.

Для исходных данных получены суммы:

$$\sum x_i^3 = -24.2, \quad \sum x_i^4 = 40.5328, \quad \sum x_i^2 y_i = -4.622582.$$

После решения системы нормальных уравнений:

$$b_0 = 0.019639, \quad b_1 = 0.550138, \quad b_2 = 0.206951.$$

Следовательно,

$$\varphi_2(x) = 0.019639 + 0.550138x + 0.206951x^2.$$

Коэффициент детерминации:

$$R_2^2 = 0.989729.$$

По значению R_2^2 квадратичная аппроксимация очень хорошая.

Среднеквадратичное отклонение:

$$\sigma_2 = 0.011512.$$

3.4 Сравнение результатов

Аппроксимация	Уравнение	σ	Вывод
Линейная	$\varphi_1(x) = -0.104532 + 0.136235x$	0.074011	удовлетворительная
Квадратичная	$\varphi_2(x) = 0.019639 + 0.550138x + 0.206951x^2$	0.011512	лучшая

Наименьшее среднеквадратичное отклонение у квадратичной модели, следовательно, именно она является наилучшим приближением для данных задачи.

3.5 Графики аппроксимирующих функций

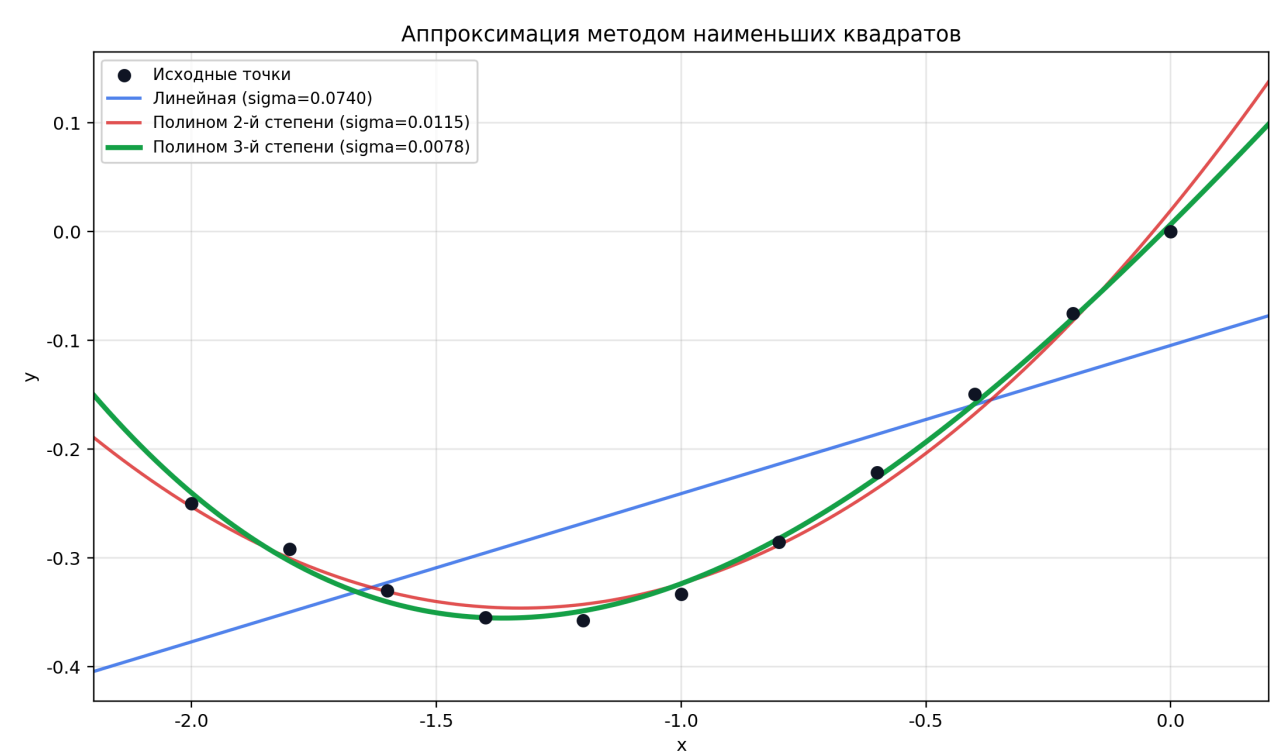


Рис. 1. График исходной функции и аппроксимирующих кривых

4 Результат выполнения программы

4.1 Тестовый набор данных (из файла)

Для демонстрации работы программы использован файл `data.csv` с 8 точками:

i	x_i	y_i
1	0	0
2	1	1
3	2	3
4	3	3
5	4	5
6	5	5
7	6	14
8	7	12

4.2 Доступность моделей

На данном наборе данных удалось построить 3 модели из 6:

- линейную,
- полиномиальную 2-й степени,
- полиномиальную 3-й степени.

Не удалось построить:

- экспоненциальную ($y > 0$ нарушается, так как есть $y = 0$),
- логарифмическую ($x > 0$ нарушается, так как есть $x = 0$),
- степенную ($x > 0$ и $y > 0$ нарушаются).

4.3 Сводная таблица результатов

Модель	Уравнение	R^2	σ
Линейная	$y = -1.166667 + 1.869048 x$	0.824850	1.973410
Полином 2-й степени	$y = 0.291667 + 0.410714 x + 0.208333 x^2$	0.865843	1.727104
Полином 3-й степени	$y = 0.318182 + 0.341270 x + 0.234848 x^2 - 0.002525 x^3$	0.865865	1.726967

4.4 Выбор наилучшей модели

Критерий выбора: минимальное среднеквадратичное отклонение σ .

$$\sigma_{\text{лин}} = 1.973410, \sigma_{\text{полином 2}} = 1.727104, \sigma_{\text{полином 3}} = 1.726967.$$

Наилучшей на данном наборе оказалась **полиномиальная модель 3-й степени**.

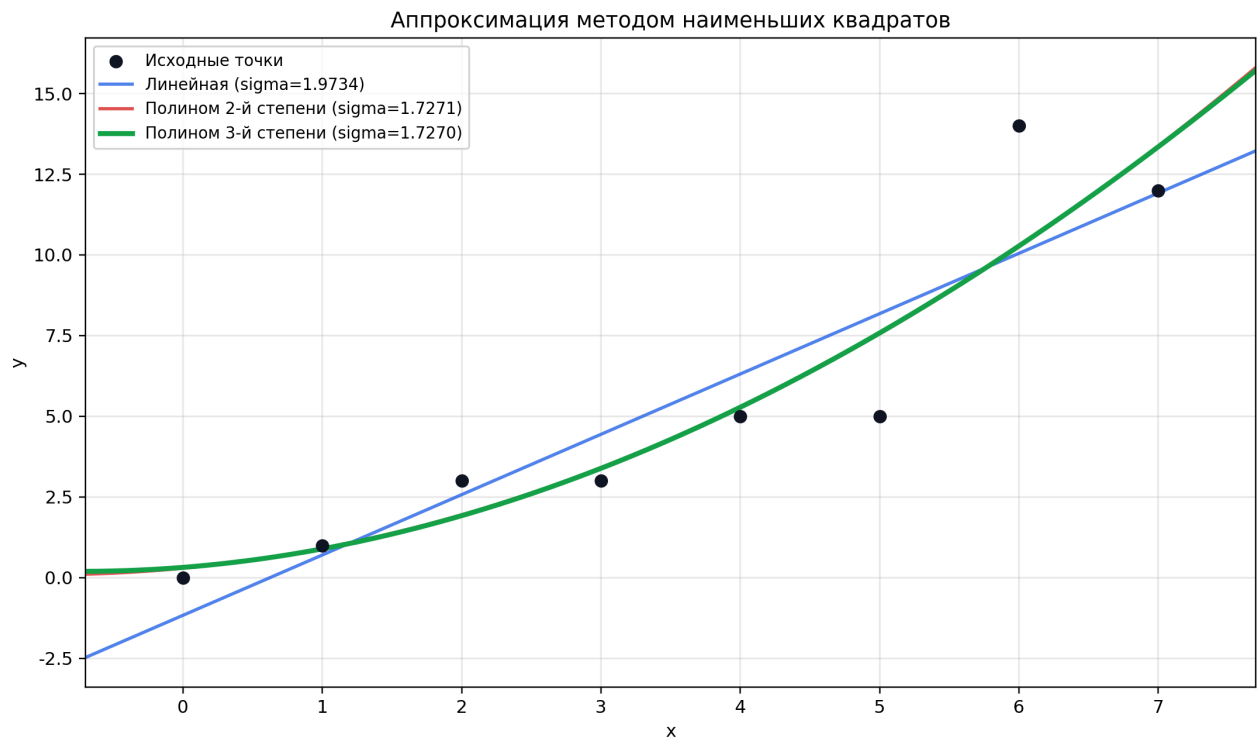


Рис. 2. График исходной функции и аппроксимирующих кривых

5 Вывод

В ходе лабораторной работы была построена табличная функция на 11 узлах и выполнена аппроксимация методом наименьших квадратов. Для варианта 8 получены линейная модель $\varphi_1(x) = -0.104532 + 0.136235x$ и квадратичная модель $\varphi_2(x) = 0.019639 + 0.550138x + 0.206951x^2$.

Сравнение по среднеквадратичному отклонению показало, что квадратичная аппроксимация заметно точнее линейной: $\sigma_2 = 0.011512$ против $\sigma_1 = 0.074011$. Коэффициент корреляции Пирсона для исходных данных составил $r = 0.758575$, а коэффициент детерминации для квадратичной модели достиг $R^2 = 0.989729$.

Таким образом, цель работы достигнута: реализован метод наименьших квадратов, рассчитаны показатели качества аппроксимации и выбрана лучшая модель.

6 Листинг программы

```
from __future__ import annotations

import csv
import math
import sys
from dataclasses import dataclass
from pathlib import Path
from typing import Callable

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```

EPS = 1e-14
MAX_ABS_INPUT = 1e50
MIN_ABS_INPUT = 1e-50

@dataclass(frozen=True)
class ApproximationResult:
    name: str
    equation: str
    coefficients: tuple[float, ...]
    rmse: float
    r_squared: float
    message: str
    predictions: list[float]
    residuals: list[float]
    predict: Callable[[float], float]
    pearson: float | None = None

@dataclass(frozen=True)
class FailedModel:
    name: str
    reason: str

def source_function(x: float) -> float:
    return 3.0 * x / (x**4 + 8.0)

def generate_variant_8_data() -> tuple[list[float], list[float]]:
    xs = [round(-2.0 + 0.2 * index, 10) for index in range(11)]
    ys = [source_function(x) for x in xs]
    return xs, ys

def normalize_number_input(raw: str) -> str:
    return raw.strip().replace(",", ".").replace("-", "~").replace("_", "~")

def parse_finite_float(raw: str) -> float:
    value = float(normalize_number_input(raw))
    if not math.isfinite(value):
        raise OverflowError("Число выходит за диапазон float.")
    if value != 0.0 and abs(value) < MIN_ABS_INPUT:
        raise OverflowError(
            f"Слишком маленькое по модулю число. Допустимо |x| = 0 или |x| >= {MIN_ABS_INPUT:.0e}."
        )
    if abs(value) > MAX_ABS_INPUT:
        raise OverflowError(
            f"Слишком большое по модулю число. Допустимо |x| <= {MAX_ABS_INPUT:.0e}."
        )
    return value

def read_float(prompt: str) -> float:

```

```

while True:
    raw = input(prompt)
    try:
        return parse_finite_float(raw)
    except ValueError:
        print("Невалидный ввод: нужно ввести число. Попробуйте еще раз.")
    except OverflowError as exc:
        print(f"Невалидный ввод: {exc} Попробуйте еще раз.")

def read_int(prompt: str, min_value: int, max_value: int) -> int:
    while True:
        raw = input(prompt).strip()
        if raw.isdigit():
            value = int(raw)
            if min_value <= value <= max_value:
                return value
        print(f"Невалидный ввод: требуется целое число от {min_value} до {max_value}.")

def input_points_from_console() -> tuple[list[float], list[float]]:
    print("\nВвод данных из консоли")
    n = read_int("Введите количество точек (8..12): ", 8, 12)
    xs: list[float] = []
    ys: list[float] = []

    for index in range(n):
        print(f"Точка #{index + 1}")
        x_value = read_float(" x = ")
        y_value = read_float(" y = ")
        xs.append(x_value)
        ys.append(y_value)

    return xs, ys

def load_points_from_file(path: Path) -> tuple[list[float], list[float]]:
    xs: list[float] = []
    ys: list[float] = []

    with path.open("r", encoding="utf-8-sig", newline="") as file:
        reader = csv.reader(file, delimiter=";")
        for row_number, row in enumerate(reader, start=1):
            if len(row) == 1:
                row = [part for part in row[0].replace("\t", " ").split(" ") if part]
            if len(row) < 2:
                continue

            try:
                x_value = parse_finite_float(row[0])
                y_value = parse_finite_float(row[1])
            except (ValueError, OverflowError):
                print(f"Предупреждение: строка {row_number} пропущена (невалидные числа).")
                continue

            xs.append(x_value)

```



```

        ys.append(y_value)

    if not (8 <= len(xs) <= 12):
        raise ValueError(
            "Таблица должна содержать от 8 до 12 корректных точек. "
            f"Сейчас получено: {len(xs)}."
        )

    return xs, ys

def select_input_data(use_cli_argument: bool = True) -> tuple[list[float],
list[float]]:
    if use_cli_argument and len(sys.argv) > 1:
        candidate = Path(sys.argv[1])
        try:
            xs, ys = load_points_from_file(candidate)
            print(f"Данные загружены из файла: {candidate}")
            return xs, ys
        except Exception as exc:
            print(f"Не удалось загрузить данные из аргумента командной строки: {exc}")
            print("Переход в интерактивный выбор источника данных.")

    while True:
        print("\nИсточник данных:")
        print("1. Вариант 8 по умолчанию")
        print("2. Загрузить из файла")
        print("3. Ввести вручную")
        print("4. Выход")
        choice = read_int("Выберите пункт (1..4): ", 1, 4)

        if choice == 1:
            xs, ys = generate_variant_8_data()
            print("Используется набор варианта 8 по умолчанию.")
            return xs, ys

        if choice == 2:
            path_raw = input("Введите путь к файлу с точками: ").strip().strip('"')
            path = Path(path_raw)
            if not path.exists() or not path.is_file():
                print("Невалидный путь к файлу. Попробуйте еще раз.")
                continue
            try:
                xs, ys = load_points_from_file(path)
                print(f"Данные загружены из файла: {path}")
                return xs, ys
            except Exception as exc:
                print(f"Ошибка чтения файла: {exc}")
                print("Введите путь еще раз или выберите другой источник данных.")
                continue

        if choice == 4:
            raise SystemExit

    return input_points_from_console()

```

```

def solve_linear_system(matrix: list[list[float]], vector: list[float]) -> list[float]:
    n = len(vector)
    a = [row[:] + [rhs] for row, rhs in zip(matrix, vector)]

    for col in range(n):
        pivot = max(range(col, n), key=lambda row: abs(a[row][col]))
        if abs(a[pivot][col]) < EPS:
            raise ValueError("Вырожденная система уравнений.")
        a[col], a[pivot] = a[pivot], a[col]

        pivot_value = a[col][col]
        for j in range(col, n + 1):
            a[col][j] /= pivot_value

        for row in range(n):
            if row == col:
                continue
            factor = a[row][col]
            for j in range(col, n + 1):
                a[row][j] -= factor * a[col][j]

    return [a[row][n] for row in range(n)]

def linear_coefficients(xs: list[float], ys: list[float]) -> tuple[float, float]:
    n = len(xs)
    sum_x = sum(xs)
    sum_y = sum(ys)
    sum_x2 = sum(x * x for x in xs)
    sum_xy = sum(x * y for x, y in zip(xs, ys))

    denominator = n * sum_x2 - sum_x * sum_x
    if abs(denominator) < EPS:
        raise ValueError("Невозможно построить линейную модель: вырожденная система.")

    a1 = (n * sum_xy - sum_x * sum_y) / denominator
    a0 = (sum_y - a1 * sum_x) / n
    return a0, a1

def polynomial_coefficients(xs: list[float], ys: list[float], degree: int) -> list[float]:
    size = degree + 1
    matrix = [
        [sum(x ** (row + col) for x in xs) for col in range(size)]
        for row in range(size)
    ]
    vector = [sum((x ** row) * y for x, y in zip(xs, ys)) for row in range(size)]
    return solve_linear_system(matrix, vector)

def pearson_correlation(xs: list[float], ys: list[float]) -> float:
    mean_x = sum(xs) / len(xs)
    mean_y = sum(ys) / len(ys)
    numerator = sum((x - mean_x) * (y - mean_y) for x, y in zip(xs, ys))
    denominator = math.sqrt(
        sum((x - mean_x) ** 2 for x in xs) * sum((y - mean_y) ** 2 for y in ys)
    )

```

```

    )
    if abs(denominator) < EPS:
        raise ValueError("Коэффициент корреляции Пирсона не определен для этих данных.")
    return numerator / denominator

def determination_message(r_squared: float) -> str:
    if r_squared >= 0.95:
        return "аппроксимация отличная"
    if r_squared >= 0.75:
        return "аппроксимация хорошая"
    if r_squared >= 0.5:
        return "аппроксимация удовлетворительная"
    return "аппроксимация слабая"

def build_result(
    name: str,
    equation: str,
    coefficients: tuple[float, ...],
    xs: list[float],
    ys: list[float],
    predict: Callable[[float], float],
    pearson: float | None = None,
) -> ApproximationResult:
    predictions = [predict(x) for x in xs]
    residuals = [prediction - y for prediction, y in zip(predictions, ys)]
    sse = sum(error * error for error in residuals)
    rmse = math.sqrt(sse / len(xs))
    mean_y = sum(ys) / len(ys)
    sst = sum((y - mean_y) ** 2 for y in ys)
    r_squared = 1.0 - sse / sst if sst > EPS else 1.0

    return ApproximationResult(
        name=name,
        equation=equation,
        coefficients=coefficients,
        rmse=rmse,
        r_squared=r_squared,
        message=determination_message(r_squared),
        predictions=predictions,
        residuals=residuals,
        predict=predict,
        pearson=pearson,
    )

def fit_linear(xs: list[float], ys: list[float]) -> ApproximationResult:
    a0, a1 = linear_coefficients(xs, ys)
    pearson = pearson_correlation(xs, ys)
    predict = lambda x: a0 + a1 * x
    equation = f"y = {a0:.6f} + {a1:.6f} x"
    return build_result("Линейная", equation, (a0, a1), xs, ys, predict,
        pearson=pearson)

```

```

def fit_polynomial_degree_2(xs: list[float], ys: list[float]) -> ApproximationResult:
    b0, b1, b2 = polynomial_coefficients(xs, ys, 2)
    predict = lambda x: b0 + b1 * x + b2 * x * x
    equation = f"y = {b0:.6f} + {b1:.6f} x + {b2:.6f} x^2"
    return build_result("Полином 2-й степени", equation, (b0, b1, b2), xs, ys, predict)

def fit_polynomial_degree_3(xs: list[float], ys: list[float]) -> ApproximationResult:
    c0, c1, c2, c3 = polynomial_coefficients(xs, ys, 3)
    predict = lambda x: c0 + c1 * x + c2 * x * x + c3 * x * x * x
    equation = f"y = {c0:.6f} + {c1:.6f} x + {c2:.6f} x^2 + {c3:.6f} x^3"
    return build_result("Полином 3-й степени", equation, (c0, c1, c2, c3), xs, ys,
predict)

def fit_exponential(xs: list[float], ys: list[float]) -> ApproximationResult:
    if any(y <= 0 for y in ys):
        raise ValueError("Экспоненциальная модель требует y > 0 для всех точек.")
    transformed_ys = [math.log(y) for y in ys]
    alpha, beta = linear_coefficients(xs, transformed_ys)
    a = math.exp(alpha)
    b = beta
    predict = lambda x: a * math.exp(b * x)
    equation = f"y = {a:.6f} * e^{{b:.6f} x}"
    return build_result("Экспоненциальная", equation, (a, b), xs, ys, predict)

def fit_logarithmic(xs: list[float], ys: list[float]) -> ApproximationResult:
    if any(x <= 0 for x in xs):
        raise ValueError("Логарифмическая модель требует x > 0 для всех точек.")
    transformed_xs = [math.log(x) for x in xs]
    a, b = linear_coefficients(transformed_xs, ys)
    predict = lambda x: a + b * math.log(x)
    equation = f"y = {a:.6f} + {b:.6f} ln(x)"
    return build_result("Логарифмическая", equation, (a, b), xs, ys, predict)

def fit_power(xs: list[float], ys: list[float]) -> ApproximationResult:
    if any(x <= 0 for x in xs):
        raise ValueError("Степенная модель требует x > 0 для всех точек.")
    if any(y <= 0 for y in ys):
        raise ValueError("Степенная модель требует y > 0 для всех точек.")
    transformed_xs = [math.log(x) for x in xs]
    transformed_ys = [math.log(y) for y in ys]
    alpha, b = linear_coefficients(transformed_xs, transformed_ys)
    a = math.exp(alpha)
    predict = lambda x: a * (x**b)
    equation = f"y = {a:.6f} * x^{{b:.6f}}"
    return build_result("Степенная", equation, (a, b), xs, ys, predict)

def run_approximations(xs: list[float], ys: list[float]) ->
tuple[list[ApproximationResult], list[FailedModel]]:
    fitters: list[tuple[str, Callable[[list[float], list[float]],
ApproximationResult]]] = [
        ("Линейная", fit_linear),
        ("Полином 2-й степени", fit_polynomial_degree_2),

```

```

        ("Полином 3-й степени", fit_polynomial_degree_3),
        ("Экспоненциальная", fit_exponential),
        ("Логарифмическая", fit_logarithmic),
        ("Степенная", fit_power),
    ]

    success: list[ApproximationResult] = []
    failed: list[FailedModel] = []

    for model_name, fitter in fitters:
        try:
            success.append(fitter(xs, ys))
        except Exception as exc:
            failed.append(FailedModel(model_name, str(exc)))

    return success, failed

def print_dataset(xs: list[float], ys: list[float]) -> None:
    print("\nИсходные точки:")
    print(f"{'i':>4} {'x_i':>16} {'y_i':>16}")
    print("-" * 40)
    for index, (x, y) in enumerate(zip(xs, ys), start=1):
        print(f"{index:4d} {x:16.8f} {y:16.8f}")

def print_model_result(result: ApproximationResult, xs: list[float], ys: list[float]) -
> None:
    print(f"\n[{result.name}]")
    print(f"Уравнение: {result.equation}")
    print(f"Коэффициенты: ", ", ".join(f"{coef:.10f}" for coef in result.coefficients))
    if result.pearson is not None:
        print(f"Коэффициент Пирсона r = {result.pearson:.6f}")
    print(f"Коэффициент детерминации R^2 = {result.r_squared:.6f} ({result.message})")
    print(f"Среднеквадратичное отклонение sigma = {result.rmse:.6f}")

    print(f"{'i':>4} {'x_i':>14} {'y_i':>14} {'phi(x_i)':>14} {'eps_i':>14}")
    print("-" * 70)
    for index, (x, y, phi, eps_i) in enumerate(
        zip(xs, ys, result.predictions, result.residuals),
        start=1,
    ):
        print(f"{index:4d} {x:14.6f} {y:14.6f} {phi:14.6f} {eps_i:14.6f}")

def save_plot(xs: list[float], ys: list[float], results: list[ApproximationResult],
best: ApproximationResult) -> None:
    output_dir = Path(__file__).resolve().parent / "img"
    output_dir.mkdir(exist_ok=True)
    output_path = output_dir / "approximation.png"

    x_min = min(xs)
    x_max = max(xs)
    margin = max((x_max - x_min) * 0.1, 0.2)
    x_left = x_min - margin
    x_right = x_max + margin
    dense_xs = [x_left + (x_right - x_left) * index / 600 for index in range(601)]

```

```

plt.rcParams["font.family"] = "DejaVu Sans"
plt.rcParams["axes.unicode_minus"] = False

fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(xs, ys, color="#111827", s=42, label="Исходные точки", zorder=4)

palette = ["#2563eb", "#dc2626", "#16a34a", "#7c3aed", "#d97706", "#0891b2"]
for index, result in enumerate(results):
    valid_xs: list[float] = []
    valid_ys: list[float] = []
    for x in dense_xs:
        try:
            y = result.predict(x)
            if math.isfinite(y):
                valid_xs.append(x)
                valid_ys.append(y)
        except (ValueError, OverflowError):
            continue
    if not valid_xs:
        continue

    linewidth = 2.8 if result.name == best.name else 1.8
    alpha = 1.0 if result.name == best.name else 0.8
    color = palette[index % len(palette)]
    label = f"{result.name} (sigma={result.rmse:.4f})"
    ax.plot(valid_xs, valid_ys, color=color, linewidth=linewidth, alpha=alpha,
label=label)

ax.set_title("Аппроксимация методом наименьших квадратов")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.grid(True, alpha=0.3)
ax.legend(loc="best", fontsize=9)
ax.set_xlim(x_left, x_right)
fig.tight_layout()
fig.savefig(output_path, dpi=220)
plt.close(fig)

print(f"\nГрафик сохранен в: {output_path}")

use_cli_argument = True
while True:
    try:
        xs, ys = select_input_data(use_cli_argument=use_cli_argument)
    except SystemExit:
        print("\nРабота программы завершена.")
        break

    use_cli_argument = False
    print_dataset(xs, ys)

    results, failed = run_approximations(xs, ys)

    if failed:
        print("\nМодели, которые не удалось построить:")

```

```
    for item in failed:
        print(f"- {item.name}: {item.reason}")

    if not results:
        print("\nНе удалось построить ни одной аппроксимации для введенных данных.")
        print("Возврат в стартовое меню...\n")
        continue

    for result in results:
        print_model_result(result, xs, ys)

    best = min(results, key=lambda item: item.rmse)
    print(f"\nЛучшая аппроксимация: {best.name} (минимальное sigma = {best.rmse:.6f})")

    save_plot(xs, ys, results, best)
```